Proposal: App Developer & Maintainer Agent (Kit)

Proposed by: Atlas (operations-manager) — on behalf of Axle (engine-dev) **Date:** 2026-03-28 **Revised:** 2026-05-31 (CEO review comments applied) **Status:** Proposed

---

Problem

IDEA's educational apps (Kolibri, Nextcloud, Kiwix, etc.) are distributed on App Disks. Each **App** is defined by a `compose.yaml` that references one or more **Services** — individual Docker containers (e.g. Kolibri, MariaDB, Redis, Nginx). Each Service has its own image and version. When an upstream project releases a new version of a Service, or when a build resource used to create an IDEA-built Service image is updated (e.g. a new Kolibri Pex file), nothing currently happens — there is no agent responsible for monitoring, updating, testing, or proposing new Apps. The app repos (`app-kolibri`, `app-nextcloud`, `app-kiwix`, `app-kolibri-studio`, `app-seafile`) exist but are unattended.

Terminology used throughout this document: - **App** — a `compose.yaml` defining one or more Services; the unit deployed on an App Disk - **Service** — a single Docker container within an App (e.g. `kolibri`, `mariadb`, `redis`) - **Service image** — the Docker image a Service runs (e.g. `learningequality/kolibri:v0.15`) - **Service resource** — the upstream build input for an IDEA-built Service image (e.g. a Kolibri Pex file) - **App version** — IDEA's versioned release of an App (bumped when any of its Services are updated)

This creates two risks: schools receive outdated software with known bugs or security issues; and new apps that could benefit teachers never get evaluated or integrated.

Proposed Solution

Add a sixth operational agent — **Kit** , App Developer & Maintainer — with four responsibilities:

1. **Service version monitoring** — detect new upstream releases of Service images and Service resources; initiate the update cycle for affected Apps
2. **App updates** — update `compose.yaml` service versions, bump the IDEA App version, run compatibility tests, and create an MC task for CEO review and approval

3. **New app proposals** — identify and evaluate new Apps suitable for offline African schools
4. **Test framework** — own and maintain the shared compatibility test harness; run upgrade tests across all Apps before any release

---

Agent Design

Identity

- **Name:** Kit
- **Role title:** App Developer & Maintainer
- **Agent ID:** `app-dev` (proposed)
- **Workspace repo:** `agent-app-dev` (new — identity files and memory only, consistent with other agent repos)
- **Telegram group:** New dedicated group, same pattern as all other agents
- **MC board:** New board in the Engineering board group

Repo Structure

```
koenswings/
  agent-app-dev/          ← Kit's workspace (identity, memory,
outputs)
  app-harness/            ← Shared test harness (new repo — owned
by Kit)
  app-kolibri/
    tests/                ← Kolibri-specific tests (Kit adds this
structure)
    test-data/            ← Test data snapshots (see Open
Questions on size)
    compose.yaml
  app-nextcloud/
    tests/
    test-data/
    compose.yaml
  app-kiwix/
    tests/
    test-data/
    compose.yaml
  app-kolibri-studio/
    tests/
    test-data/
    compose.yaml
  app-seafile/
    tests/
```

```
    test-data/
    compose.yaml
```

The shared test harness (`app-harness`) provides primitives that all app test suites can depend on. Axle's engine test infrastructure (`testMode`, disk simulation) is available as a dependency — Kit does not duplicate it.

Service Version Monitoring

Kit monitors two distinct things:

Type 1 — Upstream Service image monitoring For Services that use standard Docker Hub images (e.g. `mariadb`, `nextcloud`, `redis`), Kit uses the Docker Hub public tags API (no auth required) to detect new upstream tags. A manifest file in `app-harness` (`apps/versions.yaml`) tracks the current version of every Service image across all Apps.

Type 2 — Service resource monitoring For IDEA-built Service images (e.g. the Kolibri image, built from a Pex file), the image is not on Docker Hub — it is built by Kit from a build resource. Monitoring means checking for new versions of that build resource. The build approach and monitoring strategy per Service is defined in `app.yaml` in each app repo.

Monitoring runs as an OpenClaw **cron job** (daily). When a new Service version is detected:

1. Kit identifies all Apps that include that Service
2. For each affected App, Kit creates an MC task on its own board: `Update <app-name>: <service-name> <old-version> → <new-version>`
3. Kit does not start work until the CEO moves the task to `in_progress`
4. Once initiated:
5. **Approach A (custom build):** Kit updates the external resource reference in the `Dockerfile`, rebuilds the image on the Pi, pushes to `koenswings/<service>` on DockerHub
6. **Approach B (re-tag):** Kit pulls the new upstream image, re-tags it as `koenswings/<service>:idea-<version>`, pushes to DockerHub
7. **Approach C (direct reference):** Kit updates the image tag in `compose.yaml`
8. All approaches: Kit updates `app.yaml` versions, bumps the IDEA App version, runs the compatibility test suite
9. Test results are posted as a task comment
10. If tests pass: Kit moves the task to `review`
11. If tests fail: Kit moves the task to `review` with a `[FAILING TESTS]` flag in the comment — CEO decides whether to proceed or defer
12. CEO approves by moving the task to `done`; Kit merges the change to main

Key principle: Kit creates the task; the CEO initiates the work by moving it to `in_progress`. No work starts autonomously on version updates — each update requires explicit CEO initiation.

All image builds run on the Pi — no GitHub Actions CI. Building ARM images on ARM hardware avoids cross-architecture issues and keeps production and build environments identical.

Compatibility Test Framework

Tests use the engine test infrastructure Axle has already built (testMode + disk simulation from `agent-engine-dev`). Kit adds the app-level layer on top:

- **Smoke tests** — HTTP health check against the running container
- **UI tests** — Playwright: load key pages, assert core content visible
- **Data migration tests** — for major upgrades: confirm existing data survives the upgrade
- **Offline test** — confirm the container starts and serves with no outbound network access

App-specific tests live in each app repo (`app-kolibri/tests/`). The shared harness in `app-harness` provides the scaffolding: start a test engine, dock a fixture disk, wait for the instance to reach `Running`, then hand off to the app's test suite.

Axle maintains the engine primitives; Kit maintains the app-level harness and the per-app test suites.

New App Proposals

When Kit identifies a candidate app (from the Docker Hub ecosystem, educational software registries, or Marco's field feedback), the process is:

1. Kit assesses: offline capability, container size, complexity for teachers, licence
2. `[From Kit] Opinion` cross-agent task to Marco — field viability (is this useful for African schools?)
3. If Marco concurs: Kit creates a proposal in `idea/proposals/` and creates a task on its own MC board: `New App proposal: <app-name>`
4. CEO reviews the proposal doc and initiates the task by moving it to `in_progress`
5. Kit builds the initial app repo structure, `app.yaml`, test suite, and first App Disk version
6. Kit moves the task to `review`; CEO approves by moving to `done`; Kit merges to main

Interfaces with Other Agents

Axle (Engine Dev) - Kit depends on Axle's engine test primitives (`testMode`, disk simulation) — Axle maintains these as part of the engine;

Kit imports them - When an app major version requires engine changes: `[From Kit] Feasibility` task on Axle's board — assessment of whether the engine needs to change before the app can land - When the engine changes in ways that affect app compatibility: `[From Axle] Review` task on Kit's board — Kit runs the full app test suite against the new engine version

Marco (Programme Manager) - `[From Kit] Opinion` tasks to Marco for new app field viability assessments - Kit notifies Marco (via cross-agent task or direct message) when a new app version lands, so Marco can update teacher guides and training materials - Marco's field feedback is the primary signal for new app proposals

Atlas (Operations Manager) - Atlas owns org design — proposes any structural changes to how Kit fits into the platform via proposals in `idea/proposals/` - Quality oversight without PRs: Kit posts test results, architectural notes, and significant decisions as task comments on its MC board. Atlas monitors these during normal board polling. If Atlas spots an architectural concern, it posts a comment or opens a `[From Atlas] Review` task on Kit's board before the CEO approves the task. - Atlas onboards Kit and is the first point of contact for operational issues (disk, infra, OpenClaw config)

Affected Repos / Agents

New repos: - `koenswings/agent-app-dev` — Kit's workspace - `koenswings/app-harness` — shared test harness

Existing repos with structural additions: - `app-kolibri`, `app-nextcloud`, `app-kiwix`, `app-kolibri-studio`, `app-seafile` — each gets a `tests/` and `test-data/` directory added by Kit in a bootstrap PR

Agents affected: - Axle — provides engine test primitives; receives feasibility questions on major bumps - Marco — field viability input on new apps; receives version release notifications - Atlas — monitors Kit's MC board for architectural concerns; updates org design to add Kit

Infrastructure: - New OpenClaw agent config entry (in `openclaw.json`) - New MC board in Engineering group - New Telegram group - Daily cron job for version monitoring (OpenClaw cron scheduler) - No GitHub Actions — all image builds run on the Pi by Kit

---

Open Questions

1. **DockerHub namespace.** ~~Open~~ **Decided:** `koenswings/` — personal account continues. Migrate to a dedicated org when the GitHub org name is decided (future).

2. ~~**Image builds in CI.**~~ **Decided:** No CI. All image builds run on the Pi by Kit directly. Avoids cross-architecture issues; ARM images built on ARM hardware. The existing `build-instance` script should migrate from the engine repo to `app-harness` — see compatibility matrix sub-proposal.

3. **Test data size.** App test-data snapshots (a populated Kolibri database, a Nextcloud instance with test files) could be hundreds of MB or more. Git is not appropriate for large binaries. Options: git-lfs, external object storage, or ship test data inside the Docker image itself (cleanest for offline use). This policy needs to be decided before Kit adds `test-data/` to any app repo.

4. **App disk build.** Kit updates `compose.yaml` in the app repo, but the App Disk itself is a directory structure on a physical USB drive. Is there a `build-disk` script that creates this from the repo? If not, Kit needs to design and build one as a prerequisite.

5. **Compatibility matrix format.** Which IDEA Engine version does each app version require? A machine-readable manifest (`apps/versions.yaml`) should track this so the Engine can warn when an incompatible disk is inserted. Needs a defined schema before Kit starts versioning apps.

6. **App repos' current state.** The five existing app repos may have no CI, no tests, and no standard structure. Kit should audit these and create an MC task per repo for the bootstrap work before beginning any version monitoring work.

7. **Scope of "propose new apps."** Left unbounded this could be an infinite research task. Recommend a bounded trigger: Marco identifies an educational need first; Kit is only engaged to assess technical feasibility for an app that Marco has specifically requested. Kit-initiated proposals (no Marco request) should be limited to one per quarter.

Addendum A — Test Harness Design (Axle, 2026-05-31)

This section proposes how the test harness should invoke the engine. It was added after review of the main proposal by Axle and is pending CEO approval before Kit implements it.

Problem

The main proposal says Kit "imports" Axle's engine test primitives (`testMode`, disk simulation). Importing internal helpers from another agent's repo creates a fragile cross-repo dependency — Axle would be

blocked from refactoring internals without breaking Kit. A cleaner boundary is needed.

Proposed approach: subprocess + WebSocket API

Kit's harness starts a **real engine process** on a secondary port, talks to it over the existing WebSocket API, and tears it down after each test run. This is the same interface Axle already uses in integration tests and is already stable and versioned.

```
Kit harness
  │
  ├── spawn engine process  (IDEA_ENGINE_PORT=18800
IDEA_TEST_MODE=true)
  │     └── engine listens on ws://localhost:18800
  │
  ├── wait for Ready signal over WebSocket
  │
  ├── present fixture disk  (pre-mounted directory at /disks/
<device>/)
  │
  ├── wait for instance Running state over WebSocket
  │
  ├── hand off to app test suite  (smoke, UI, migration tests)
  │
  └── send undockDisk + kill process
```

Running alongside a live engine

Kit runs on a Pi that already has a live engine on port 18789 (the production engine). The test harness must not interfere with it.

Solution: the harness spawns the test engine on a **different port** (e.g. 18800) with a **separate store directory** (e.g. `store-data-test/`). Both engines run simultaneously with no shared state.

```
# Environment variables the harness sets before spawning:
IDEA_ENGINE_PORT=18800
IDEA_STORE_DIR=/tmp/kit-test-store-$$   # unique per run
IDEA_TEST_MODE=true                     # skips sudo mount/umount,
borg, docker
IDEA_SYSTEM_DISK_SKIP=true              # do not register the Pi
boot disk
```

`IDEA_TEST_MODE=true` is already supported by the engine and skips all operations that require root or physical hardware. This is the same flag Axle uses in unit tests.

`IDEA_SYSTEM_DISK_SKIP` may need to be added to the engine (small change — prevents the test engine from registering the Pi's own boot disk as a system disk on startup).

What testMode skips vs. what runs identically to production

Before claiming the harness tests a real app in a real environment, it is important to be precise about what `testMode` actually changes.

Skipped in testMode (differs from production):

Skipped operation	Production behaviour	testMode behaviour
`sudo mount`	Mounts USB disk at `/disks/<device>/`	Fixture directory must exist pre-mounted
Hardware ID lookup	Reads serial number from block device	Uses diskId from META.yaml as-is
Image loading from tar	`docker image load < services/<image>.tar`	Docker pulls image from Hub if not cached
`sudo` writeMeta	Updates META.yaml on disk	META.yaml is not mutated
Borg backup/ restore	Runs borg commands	Skipped entirely

Identical to production (not skipped):

- `docker compose create` — real containers created
- `docker compose up -d` — real containers started
- Port assignment, `.env` generation, password generation
- All state machine transitions (Starting → Running → Stopped → Error)
- The full WebSocket/store flow
- App-specific pre-processing (e.g. hostname/IP injection for Nextcloud)

Conclusion: The app starts as real Docker containers, driven by a real engine, via the same code path as production. The environment is authentic at the Docker and engine level.

Fixture disk — use real app repo content

Because `sudo mount` is skipped, the harness must prepare a fixture directory at `/disks/<device>/` before the test engine starts. This directory must mirror the structure of a real App Disk:

```
/disks/sdb1/                    ← fixture root (created by harness)
  META.yaml
  apps/
    kolibri-1.0/
      compose.yaml              ← taken directly from app-kolibri
repo
  instances/
    <instanceId>/
      compose.yaml
      .env
```

The fixture must use the real `compose.yaml` from the app repo — not a stub or simplified version. This ensures the containers, image references, volumes, and env variables are identical to what a teacher would receive on a real App Disk.

Known gaps vs. production

Two aspects of the production path are not exercised by the harness:

Gap 1 — Image tar loading. In production, Docker images are pre-bundled as `.tar` files on the disk and loaded with `docker image load`. In testMode, Docker pulls from the internet instead. The tar-loading path is therefore not tested by the harness. This is acceptable for automated testing on a Pi with internet, but means a regression in the tar-loading code would not be caught by Kit's test suite.

Mitigation: Axle covers this path in engine unit tests. Additionally, a manual "production path smoke test" (dock a real App Disk, confirm it starts) should be run once per app before the first release — not automated.

Gap 2 — Physical disk mount. The harness never exercises `sudo mount` or the USB device detection path. A failure in mount logic would not be caught.

Mitigation: This is Axle's responsibility, covered in engine integration tests. Kit does not need to test it.

Axle's commitments

- Keep `testMode` and the WebSocket command API stable and backwards-compatible
- Keep `IDEA_ENGINE_PORT` and `IDEA_STORE_DIR` env-variable overrides working
- Add `IDEA_SYSTEM_DISK_SKIP` if not already present
- When breaking changes are unavoidable: open `[From Axle] Review` task on Kit's board before merging

What Kit owns

- The harness bootstrap/teardown logic (spawn, wait, connect, kill)
- Fixture disk directories built from real app repo content
- All app-level tests (smoke, UI, migration, offline)
- The one-time manual production path smoke test per app

What Kit does NOT own

- Engine internals — no direct imports from `agent-engine-dev/src/`
- The WebSocket API schema — Kit reads it, Axle defines it
- The mount/USB detection path — Axle tests this

Addendum B — Cross-Agent Task Message Template (Axle, 2026-05-31)

Proposed standard for all cross-agent tasks posted to another agent's MC board.

Title format

```
[From <Sender>] <Type>: <short description>
```

Types:

Type	Meaning
`Feasibility`	Sender needs receiver to assess whether something is buildable
`Review`	Sender has changed something; receiver must re-test or re-assess
`Opinion`	Sender wants receiver's field/domain judgement before proceeding
`Done`	Sender has completed work the receiver was waiting on
`FYI`	No action required; informational only

Description template

```
**From:** <AgentName> (<agent-id>)
**Date:** YYYY-MM-DD
**Type:** <type from table above>
**Waiting on reply:** yes | no
```

```
---

<Self-contained description of what is needed and why. Include
enough context
that the receiving agent can act without reading the full
conversation history.>

## What I need from you

<Concrete, bounded ask. One thing if possible.>

## Background

<Any relevant prior decisions, PR links, or task IDs.>
```

Rules

1. **Self-contained** — the description must make sense without external context.
2. **One ask per task** — if two separate actions are needed, open two tasks.
3. **Depth-1 only** — do not open a cross-agent task in response to a cross-agent task; relay through Koen if a chain is forming.
4. **Reply by comment** — the receiving agent posts their answer as a task comment, then optionally opens a `[From <Receiver>] Done` task on the sender's board.
5. **Tag: cross-agent** — always add this tag so boards stay scannable.

Addendum C — Modelling Kit's Work in Mission Control (Axle, 2026-05-31)

Proposed MC board structure and task flow for Kit's onboarding and ongoing work.

Boards

Board	Owner	Group
Engine Dev	Axle	Engineering
Console Dev	Pixel	Engineering
App Dev	Kit	Engineering
Operations	Atlas	Operations
Programme	Marco	Programme

Kit gets a new **App Dev** board in the Engineering group, same pattern as existing agents.

Onboarding tasks (one-time)

These tasks exist to bring Kit into existence. They live on Atlas's board (Atlas owns org-level setup) except where noted.

Board	Title	Owner
Atlas	Create Kit agent: repo, openclaw.json, MC board, Telegram group	Atlas
Atlas	Bootstrap app repos: add tests/, test-data/, app.yaml to each	Atlas
Axle	Add requires_engine_min compatibility check on disk dock	Axle
Kit (once created)	Write harness: subprocess engine + WebSocket bootstrap/teardown	Kit
Kit	Write smoke + UI tests for each of the 5 apps	Kit
Kit	Write app.yaml for each of the 5 apps	Kit

Ongoing task patterns

Daily Service version monitoring cycle (Kit-initiated):

```
Kit detects new Service version (Docker Hub tag or new build
resource)
  → Kit creates MC task: "Update <app>: <service> <old> → <new>"
  → Task sits in inbox — no work starts yet
  → CEO reviews and moves task to in_progress to initiate
  → if major Service bump: [From Kit] Feasibility task on Axle's
board first
  → Axle assesses, posts comment
  → Kit updates compose.yaml, bumps App version, runs tests
  → Kit posts test results as task comment
  → Kit moves task to review
  → CEO approves → moves to done
  → Kit merges to main
```

Engine breaking change (Axle-initiated):

```
Axle merges a breaking engine change
  → Axle opens [From Axle] Review task on Kit's board
```

```
  → Kit re-runs full app test suite
  → Kit posts results as comment
  → If failures: Kit opens [From Kit] Feasibility on Axle's board
  → Chain resolved; Kit moves Review task to done
```

New App proposal (Marco-initiated):

```
Marco identifies educational need
  → [From Marco] Opinion task on Kit's board
  → Kit assesses technical feasibility
  → Kit posts comment + writes proposal in idea/proposals/
  → Kit creates MC task: "New App proposal: <app-name>"
  → CEO reviews proposal doc and moves task to in_progress to
approve
  → Kit builds app repo, app.yaml, test suite, first App Disk
  → Kit moves task to review
  → CEO moves to done → Kit merges to main
```

What lives where

- **MC tasks** — all work, including cross-agent triggers
- **Task comments** — inter-agent replies, branch names, test results
- **Proposal docs** (`idea/proposals/`) — architectural decisions requiring CEO approval
- **No direct agent-to-agent messaging** — everything routes through MC or Koen

---

Addendum D — Operations Review (Atlas, 2026-05-31)

Review of the full proposal and Axle's addenda from the operations angle, against the current MC-native platform. Pending CEO approval before implementation begins.

What changed since March 2026

The proposal was written under the old GitHub-PR-centric workflow. Since the MC-native migration (2026-05-30), the operational model has shifted:

- **No PR gatekeeper role for Atlas.** Agents with direct push access to their own repos (Axle, Pixel) don't route through Atlas for merges. Kit should follow the same pattern: direct push access to `agent-app-dev`, `app-harness`, and the five app repos. Atlas reviews proposals and org design, not individual PRs.

- **CEO approval via MC, not GitHub merge.** The proposal currently says "CEO approves by merging the proposal PR." This should be: CEO moves the relevant MC task to `done`. A merge to main may still happen, but it's the implementation step, not the approval gate.
- **No BACKLOG.md.** The MC board is the backlog. Kit's inbox is its task queue. No `BACKLOG.md` file in `agent-app-dev`.

Structural observations

Addendum A (Test Harness) — Approved. The subprocess + WebSocket approach is correct for cross-repo stability. One operational note: Kit's test runs will be resource-heavy on the Pi (spawning a second engine process, running containers). Kit should not run tests during production engine operations. The harness should check whether any App Disk is currently running before spawning a test engine, and abort + reschedule if so. Axle owns the engine API that exposes running state; Kit reads it.

Addendum B (Cross-Agent Task Template) — Approved with one addition. The five task types and the description template are clean. Adding one rule: tasks of type `Feasibility` and `Review` that remain unanswered for more than 5 days should be escalated by the sender via Telegram to Koen. MC does not have SLA enforcement; the sender agent's cron poll handles escalation.

Addendum C (MC Board Structure) — Approved. The board layout and task flow diagrams are correct. One correction to the onboarding task table: the `Bootstrap app repos` task belongs on **Kit's board** once Kit exists, not Atlas's. Atlas creates Kit; Kit bootstraps its own app repos. Atlas's onboarding work is scoped to: (1) create the GitHub repo, (2) add the OpenClaw agent config entry, (3) create the MC board, (4) set up the Telegram group. Everything after that is Kit's.

Atlas's onboarding task list (updated)

Task	Board	Description
Create `agent-app-dev` repo on GitHub	Atlas	Apply the standard agent repo template
Add Kit to `openclaw.json`	Atlas	New agent entry, `app-dev` ID, workspace path
Create App Dev board in MC	Atlas	Engineering group, same config as existing boards
Create Kit's Telegram group	Atlas	Get chat ID from Koen; add bot
Generate Kit's `AUTH_TOKEN`	Atlas	Via `setup.sh` DB write pattern

Task	Board	Description
Bootstrap Kit's identity files	Atlas	`AGENTS.md`, `SOUL.md`, `TOOLS.md`, `MEMORY.md` stubs

These six tasks constitute the full onboarding scope for Atlas. All subsequent work (harness, test suites, `app.yaml` per app, bootstrap work per app repo) is Kit's.

Open questions not yet resolved

1. **DockerHub namespace** — `koenswings` (personal) vs a dedicated org. This decision should be made before Kit builds any images. Recommend Koen decides at the same time as approving this proposal.
2. ~~**GitHub Actions on app repos.**~~ **Decided:** No GitHub Actions. All image builds run on the Pi by Kit. Kit audits each app repo's current state in its first bootstrap pass.
3. **Test data storage** — large binary snapshots (Kolibri DB, Nextcloud files) don't belong in git. Recommend a decision: git-lfs or ship test data inside a dedicated Docker test-data image. Lean toward the Docker image approach — consistent with the offline-first philosophy and avoids git-lfs hosting costs.

Summary

The proposal is well-scoped and all three addenda from Axle are sound. The two doc updates needed before approval: (1) remove Atlas as PR gatekeeper — Kit has direct push access to its own repos; (2) replace "CEO approves by merging" with "CEO moves MC task to done." Everything else is implementation-ready pending CEO decision.